

Week 5 Assignment – Apache Spark Fundamentals

Name : Siga Nithin

College : CVR College Of Engineering

StudentId : CT_CSI_DE_649

Introduction

This assignment helped me understand the basics of Apache Spark and its DataFrame API for processing data efficiently. I learned how to load CSV data, clean duplicate and missing records, apply filtering and transformations, and perform aggregation operations. Through this assignment, I also gained a basic understanding of Spark concepts such as in-memory computing, immutability, and shuffle operations.

Week 5 – Theory Questions

Note: The practical implementation and code for the remaining questions have been completed in the submitted PySpark notebook.

Limitations of MapReduce and Advantages of Spark

- MapReduce stores intermediate data on disk after every processing stage, making execution slower.
- It is not suitable for iterative tasks such as machine learning.
- Complex workflows require multiple jobs.
- Spark processes data in memory, making execution much faster.
- Spark also supports SQL, Streaming, Machine Learning, and Graph processing using a unified framework.

In-Memory Computing in Spark

- Spark stores intermediate data in RAM instead of writing it to disk.
- Data can be reused across multiple iterations without repeated disk access.
- This significantly improves the performance of machine learning and data analysis applications.
- It reduces execution time and improves overall efficiency.

Difference between .na.drop() and .na.fill()

- `.na.drop()` removes rows that contain null values, whereas `.na.fill()` replaces null values with a specified value.
- Using `.na.drop()` may reduce the size of the dataset, while `.na.fill()` helps preserve useful records.
- The choice between these methods depends on the importance and meaning of the missing data.

Immutability of Spark DataFrames

- Spark DataFrames are immutable, meaning the original DataFrame is never modified.
- Every transformation creates a new DataFrame instead of updating the existing one.
- This makes data processing safer, more reliable, and easier to debug.

Importance of Handling Null Values Before Aggregation

- Null values can affect calculations such as `sum()` and `avg()`.
- Handling missing values before aggregation improves data quality and produces more accurate analytical results.
- Proper preprocessing ensures reliable reports and analysis.

Shuffle Operation in Spark

- Shuffle is the process of moving data between Spark partitions during operations such as `groupBy()` and `join()`.
- Since data is exchanged across partitions, it is considered a wide transformation.
- Shuffle increases execution time, so reducing unnecessary shuffle improves Spark performance.

Risk of Using `inferSchema=True`

- Spark may infer incorrect data types when the dataset contains inconsistent date formats.
- Some values may become null due to incorrect schema detection.

- Defining the schema explicitly is a better practice for production environments.

Challenges Faced and How I Overcame Them

- **Handling Missing Values:** I carefully decided whether to fill missing values or remove records based on the importance of the data to avoid unnecessary data loss.
- **Understanding Shuffle Operations:** I used the execution plan to understand why `groupBy()` is slower than `filter()` and learned how shuffle affects performance.
- **Understanding DataFrame Immutability:** I learned that every transformation creates a new DataFrame instead of modifying the original one, which makes Spark pipelines more reliable.

Conclusion

This assignment helped me understand the fundamentals of Apache Spark and its DataFrame API. I learned how to clean, transform, filter, and analyze datasets using PySpark while gaining practical knowledge of concepts such as in-memory computing, immutability, and shuffle operations. Overall, this assignment strengthened my understanding of Spark-based data processing workflows.